AI Model Compiler & Optimization Engine

Implementation status and code update report | 2026-09-25

Executive status

The repository now contains a tested compiler core for importing selected PyTorch models into a custom intermediate representation, validating the graph, applying graph optimizations, executing the result through a reference backend, and measuring latency. The work is a functional prototype; the full research program is not yet complete.

Area	Current status
Custom IR	Implemented and validated
PyTorch importer	MLP and controlled CNN subset implemented
Optimization passes	Constant folding, dead-node, redundant-op, and Conv-BN-ReLU fusion
Reference backend	Implemented with PyTorch tensor kernels
Precision	FP16 and symmetric weight-only INT8 implemented
Visualization	DOT output implemented
Benchmarking	Repeated latency and throughput runner implemented
GUI	Tkinter workbench for compile, benchmark, and IR export
Automated tests	9 tests passing at report generation

1. Architecture implemented

The current pipeline is:

```
PyTorch model -> torch.fx extraction -> custom Graph/Node/Tensor IR -> validation ->
optimization pass -> reference backend -> numerical comparison and benchmark
```

2. Code updates made

The following modules were added or expanded during implementation:

- compiler/ir: graph ownership, tensor metadata, producer/consumer links, boundary checks, topological ordering, and cycle detection.
- compiler/frontend/pytorch_importer.py: FX import for Linear, ReLU, Conv2d, BatchNorm2d, MaxPool2d, AvgPool2d, Flatten, Add, and Mul.
- compiler/optimizer: pass manager, constant folding, dead-node elimination, scalar identity elimination, and inference Conv-BatchNorm-ReLU fusion.
- compiler/backend/reference.py: execution of the imported and fused IR using PyTorch reference kernels.
- compiler/quantization/precision.py: FP16 conversion plus symmetric weight-only INT8 quantization with retained scales.
- visualization/dot.py: Graphviz DOT generation without requiring Graphviz installation.
- benchmarks/runner.py: warmups, repeated timings, mean/median/p95 latency, throughput, and runtime metadata.
- tests/: import, execution, malformed operator diagnostics, optimization, fusion, precision, visualization, and benchmark regression tests.
- compiler/gui.py: Tkinter workbench for selecting workloads, precision, passes, benchmarking, and exporting IR JSON.

3. Correctness evidence

The most important checks compare reference PyTorch execution against execution of the custom IR. The MLP and CNN checks pass with zero maximum error in the tested FP32 cases. Conv-BatchNorm-ReLU fusion is checked with dtype-appropriate tolerance, and the FP16 path verifies reduced-precision output and constant metadata.

```
Latest validation result: 12 tests passed; static diagnostics reported no errors; Python
bytecode compilation completed successfully.
```

4. Benchmarking currently available

The benchmark runner is a measurement utility. It records warmup count, run count, batch size, mean latency, median latency, p95 latency, throughput, Python version, PyTorch version, and device. It does not claim an optimization speedup until controlled baseline and optimized experiments are run.

```
Run: python -m scripts.benchmark_demo
```

5. Important design decisions

The project owns the IR and optimization logic. PyTorch FX is used for extraction and PyTorch kernels are used only as a transparent reference backend. This preserves the compiler research focus while avoiding premature hand-written numerical kernels. Unsupported operators raise explicit diagnostics instead of silently producing an incomplete graph.

6. Current limitations

The current repository is not yet the complete final research system. It does not yet include INT8 activation calibration, ONNX export or runtime comparison, peak-memory measurement, serialized model-size measurement, accuracy evaluation on CIFAR-10, ResNet-18 or MobileNet experiments, automated ablation reports, benchmark plots, or the final academic report.

7. Recommended next updates

The next implementation sequence should be:

- Add graph serialization and stable experiment configuration files.
- Add an INT8 investigation with calibration, numerical-error reporting, and explicit accuracy trade-offs.
- Add benchmark result persistence to CSV/JSON and generate plots for baseline versus each pass.
- Add ResNet-18 and MobileNet controlled workload import tests, then run ablation experiments.
- Add optional ONNX Runtime comparison while keeping custom IR transformations independent.
- Write the research report from measured results, including neutral or failed optimization cases.

8. Reproduction commands

From the repository root:

```
python -m pytest -q python -m compiler.demo python -m scripts.benchmark_demo python -m
compileall -q compiler benchmarks visualization scripts tests
```

Source references

The living technical plan is in docs/PROJECT_BLUEPRINT.md. The implementation overview and setup commands are in README.md. This PDF is generated by docs/generate_status_pdf.py so it can be refreshed as the code changes.